

Formal Review Request Template

Requester: Javier Nieves

Date: 10/06/2026

Reviewer: Sri Aashrith Vithana

Approx. review duration: 1 hour

Choose one primary review method:

Code / Software Review

Code Quality Review

Inspect the code for issues such as readability, modularity, maintainability, efficiency, and appropriate programming practices.

White-Box / Edge-Case Testing

Inspect the internal logic and systematically test branches, boundary conditions, exceptions, or adversarial inputs.

Black-Box / Functional Testing

Develop test cases from the expected requirements or behavior and determine whether the software produces the expected results.

Performance / Runtime Testing

Systematically evaluate whether the software meets relevant runtime or performance expectations.

Usability Review

- **Task-Based Usability Testing**

Give the reviewer/user specific tasks to complete while you observe what they do, where they succeed, and where they encounter problems. A think-aloud protocol may be used.

- **Heuristic / Guidelines Evaluation**

Systematically inspect the interface using identified usability principles, heuristics, or design guidelines.

- **Prototype Usability Testing**

Have users complete defined tasks using a wireframe, prototype, or partially implemented interface.

Design Review

- **Architecture / Conceptual Design Review**

Evaluate whether architecture and major design decisions align with project requirements and constraints.

- **Preliminary Design / UML Review**

Evaluate proposed system structure, diagrams, component relationships, interfaces, or other design artifacts.

- **Security Design Review**

Evaluate relevant authentication flows, trust boundaries, data protection, error handling, or other security-related design decisions.

Other

- **Other review method:**

Explain what the reviewer will evaluate and why this method is appropriate for your work:

Login as a Admin, Create a semester, assign students, see progress they have made.

Review Request Details

1) What specifically should the reviewer examine or interact with?

The Status Page and see if information is accurate

2) What question do you want this review to help answer?

Examples:

- Does this implementation correctly handle boundary and error cases?
- Can a new user complete the checkout task without assistance?
- Does this architecture satisfy our project's stated requirements?

Is creating a new semester simple?

Is adding students straightforward?

Are you able to see useful information for each team?

Is any information not useful?

3) What should the reviewer focus on?

Primarily on the status page

4) What should the reviewer *not* evaluate during this review?

Examples:

- Do not evaluate visual polish or mobile responsiveness (for a usability test evaluating navigation and task completion)
- Do not evaluate the database layer or unfinished API integration (for a code review focusing on the authentication module)

The actual code behind the app, just focus on how it handles, and see if its easy to use.

5) What should the reviewer use to judge the work?

Examples include expected software behavior, project requirements, coding best practices, usability heuristics, or design constraints.

If the design is easy to follow like any professional app, and how easy it is to use as a student.

Reviewer Section

1) Briefly describe what you actually did during the review.

Examples include executing defined test cases, inspecting particular code paths, observing a user completing defined tasks, or systematically examining a design against specified criteria.

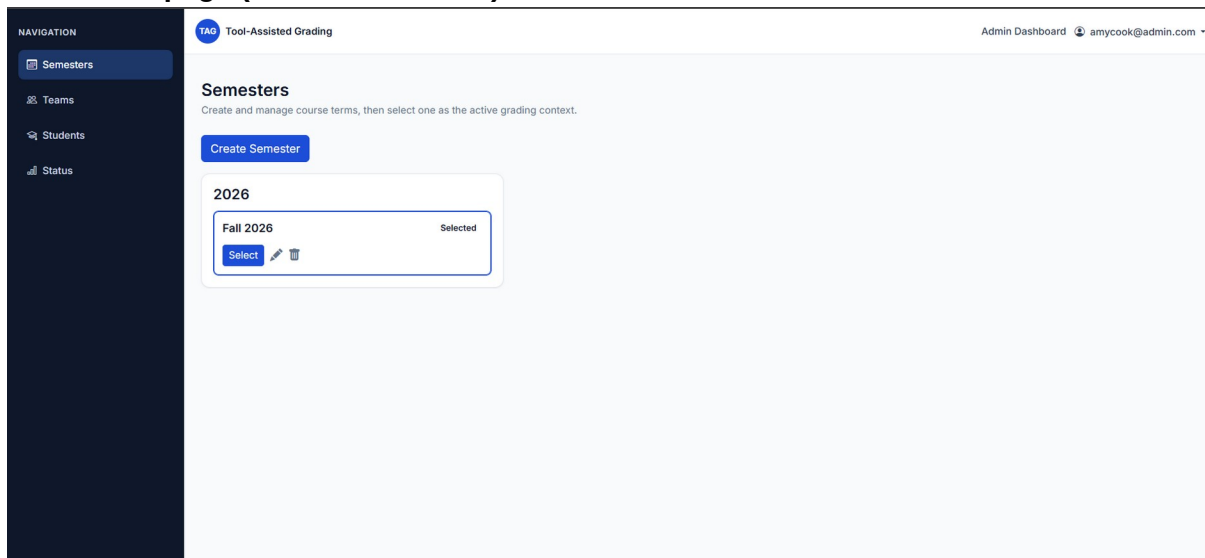
I logged in as an admin and did the tasks I was given. I created a new semester (Fall 2026), made a team, added a student and tried to assign him to that team. Then I went to the Status page to see if the team information showed up correctly.

2) What portions of the artifact did you evaluate?

The Semesters, Teams, Students and Status pages. I spent most of my time on the Status page like you asked.

3) Provide or link to relevant evidence such as test cases, screenshots, notes, annotated code, observations, or other review artifacts.

Semesters page (semester created):



Teams page:

NAVIGATION

Semesters

Teams

Students

Status

TAG

Tool-Assisted Grading

Admin Dashboard amycook@admin.com

Teams

Review team membership and core project links for the selected semester.

New Team

Import Team Roster CSV

Upload a roster CSV to create/update students and teams for the selected semester.

Choose File No file chosen

Import CSV

Team Overview

TEAM NAME	MEMBERS	GITHUB TOKEN	LINKS	ACTIONS
sun	dave@gmail.com		RepositoryProject Board TimesheetClient Notes	

Students page (student added to team "sun"):

NAVIGATION

Semesters

Teams

Students

Status

TAG

Tool-Assisted Grading

Admin Dashboard amycook@admin.com

Students

Manage roster records and contact links for the selected semester.

New Student

Roster

FULL NAME	EMAIL	GITHUB USERNAME	TEAM	LINKS	ACTIONS
dave	dave@gmail.com	dave@github	sun	EmailGitHub	

Status page:

NAVIGATION

Semesters

Teams

Students

Status

TAG Tool-Assisted Grading

Admin Dashboard amycook@admin.com

Attention: No Student CSV has been uploaded for this semester. Please upload the student CSV to display the data.

Team Status

Update Cached Status Data

Status snapshot built at Oct 7, 2026 9:33 PM CT. Refreshing without the update button should reuse this snapshot.

Sprint: Sprint 1 Sprint 2 Sprint 3 Sprint 4

Stage: Planning Progress Check Demo Day

sun

Team Evaluation

Feature Selection & Decomposition	Feature Assignments	Kanban Updates
B: 0 · T: 0 · IP: 0 · D: 0	0	No data available

Individual Evaluation

Member	Time Estimate
dave d.	0

Findings

Strengths:

- Creating a new semester was simple and quick. The "Selected" label makes it clear which semester is active.
- The sidebar makes it easy to move between pages, and every page is clean and easy to read.
- The Status page layout is nice. Picking a sprint and stage with the buttons is easy, and splitting team and individual evaluation makes sense.

Issues / Defects:

Description of Issue	Severity (Low/Med/High)	Evidence / Example	Suggestions
Assigning students to a team was confusing	MED	I was not sure if I should do it on the Students page, the Teams page, or with the CSV import. It took me a while to figure out.	Add a simple "Team" dropdown when adding a student, or a short note on the page that says where to assign students.

Status page info is not accurate: it says no student CSV was uploaded	MED	The red banner shows up even though I already added a student (dave) to team "sun".	Only show this message when there are really no students in the semester.
Some info on the Status page is hard to understand	LOW	"B: 0 · T: 0 · IP: 0 · D: 0" has no labels, so I did not know what the letters mean. Kanban Updates just says "No data available."	Spell out the words (Backlog, To Do, In Progress, Done) or add a hover tooltip, and say why there is no data.

Requester Follow-Up

1) How will you address the feedback? If you decide **not** to, briefly explain why.

I will address the feedback by improving the usability and clarity of the application.

First, I will make assigning students to teams easier by adding clearer instructions or a team dropdown on the Students page.

Second, I will investigate the Status page warning about missing student CSV data. I will make sure the message clearly explains when a CSV upload is required, even if students have already been added manually.

Finally, I will make the Status page easier to understand by replacing abbreviations like B, T, IP, and D with full labels (Backlog, To Do, In Progress, and Done). I will also add clearer explanations when information is unavailable.

I plan to discuss these improvements with my team and prioritize them in an upcoming sprint.
